

Chronic Kidney Disease Prediction

An End-to-End Machine Learning Project

CBIO313 — Data Mining & Machine Learning
Final Project Report

1. Problem Definition and Objectives

Chronic Kidney Disease (CKD) is a progressive condition in which kidney function declines over time, often without noticeable symptoms until significant damage has occurred. Early detection from routine blood and urine tests allows clinicians to intervene sooner and slow disease progression.

Objective: Given a patient's routine clinical and laboratory measurements (age, blood pressure, blood glucose, hemoglobin, serum creatinine, and related markers), build a machine learning model that predicts whether the patient has CKD, and identify which measurements are most informative.

2. Dataset Description

Source: UCI Machine Learning Repository — Chronic Kidney Disease dataset, collected at a hospital in Tamil Nadu, India over approximately two months.

- 400 rows (patients), 25 raw columns (24 clinical features + target classification)
- Class balance: 250 CKD patients, 150 non-CKD patients
- The raw, unprocessed version of the file was used deliberately (not a pre-cleaned variant), so that data cleaning was a genuine requirement of the project rather than a formality.

Data-quality issues present in the raw file included: 1,012 missing cells spread across the 25 columns, a literal "?" placeholder used for missing values in some rows, embedded tab characters inside cell values, numeric columns stored as text because of stray tokens, and one corrupted target label (a tab-contaminated "ckd") requiring correction.

3. Data Preprocessing

The following cleaning steps were applied, in order, and documented in the project notebook:

- Stripped whitespace/tab characters from column names and every string cell.
- Standardized missing-value markers (empty strings and "?") to NaN.
- Corrected the corrupted target label back to "ckd".
- Coerced numeric-meaning columns (packed cell volume, white cell count, red cell count, etc.) that were read as text back to numeric dtype.
- Dropped the id column (a row index, not predictive).
- Imputed missing numeric values with the column median (robust to the heavy right-skew and outliers present in lab values such as serum creatinine) and missing categorical values with the column mode.
- Encoded the target as a binary label: ckd = 1, notckd = 0.

After cleaning, the dataset contained zero missing values across all 400 rows and 24 usable feature columns.

4. Exploratory Data Analysis

EDA investigated the research question from multiple angles using both univariate analysis (distribution of a single variable) and bivariate analysis (relationship between a variable and the target), across more than six variables, using five distinct plot types: histograms, boxplots, count plots, a stacked bar chart, and a correlation heatmap.

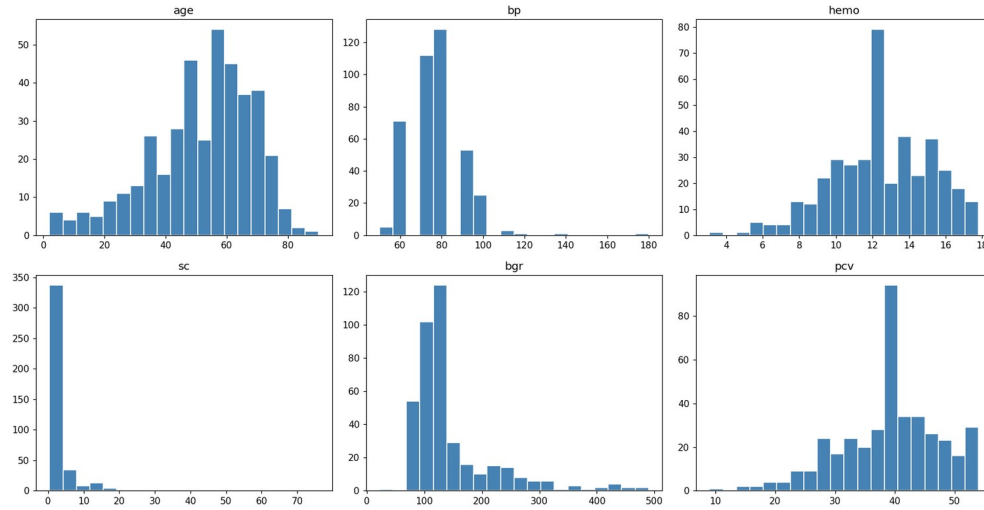


Figure 1. Univariate distributions of six numeric features.

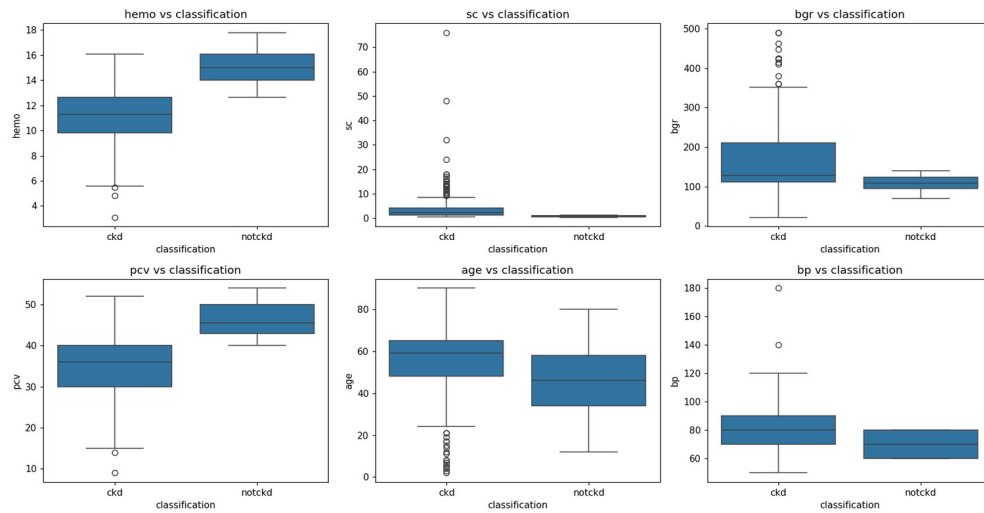


Figure 2. Bivariate comparison of six numeric features against the target class.

Hemoglobin and packed cell volume were visibly lower in CKD patients, while serum creatinine was visibly higher — consistent with clinical expectations and a useful sanity check that the cleaned data preserved a real medical signal.

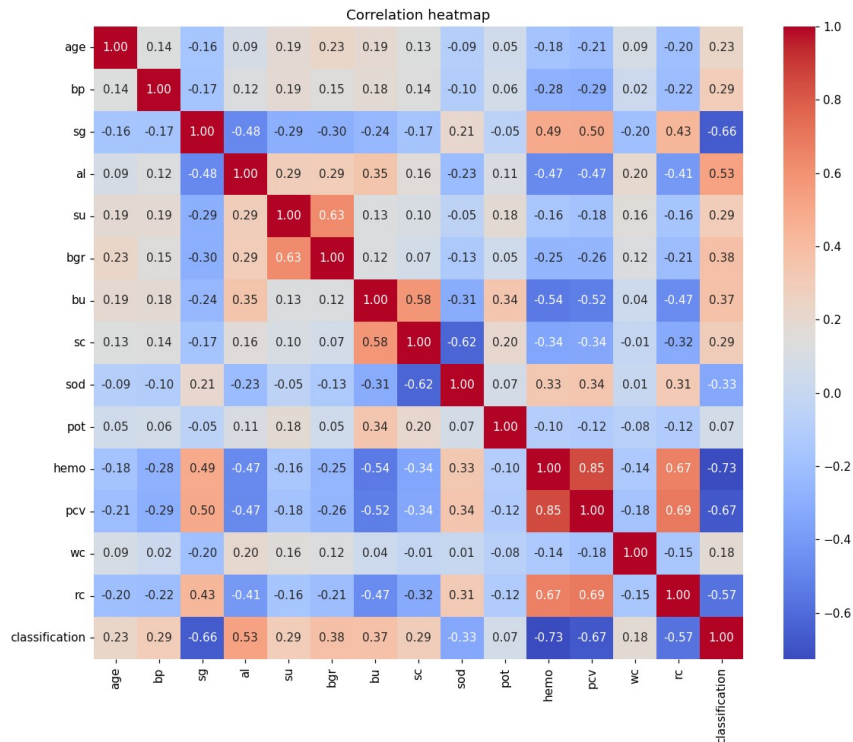


Figure 3. Correlation heatmap of numeric features and the target.

5. Feature Engineering and Selection

5.1 New feature

A new feature, bun_creatinine_ratio (blood urea nitrogen divided by serum creatinine), was engineered. In nephrology this ratio is a recognized marker used to help distinguish pre-renal from intrinsic renal causes of dysfunction, so it was added explicitly rather than left for the model to infer implicitly from the two raw columns.

5.2 Feature selection

Remaining categorical columns were label-encoded, and a filter method — SelectKBest with the ANOVA F-statistic (f_{classif}) — was used to score each feature's individual relationship with the target, independent of any specific model, keeping the top 12 features for modeling. This included specific gravity, albumin, pus cell, blood glucose, hemoglobin, packed cell volume, red cell count, hypertension, diabetes mellitus, appetite, pedal edema, and the engineered BUN/creatinine ratio.

6. Model Training, Tuning, and Validation

Three algorithms from different modeling families were trained and compared: Logistic Regression (a linear baseline), Random Forest (a bagged tree ensemble capturing non-linear interactions), and Support Vector Machine (a margin-based classifier). Each was tuned with GridSearchCV over a small hyperparameter grid using 5-fold cross-validation, optimizing F1-score.

6.1 What is hyperparameter tuning, and why does it matter?

Hyperparameters are settings chosen before training rather than learned from the data (e.g. the regularization strength C in Logistic Regression/SVM, or the number of trees and maximum depth in Random Forest). Poor choices lead to overfitting (the model memorizes training noise) or underfitting (the model is too constrained to

capture real patterns). Grid search systematically evaluates combinations of candidate values and selects the combination with the best cross-validated score, rather than relying on a single lucky train/test split.

6.2 What is validation, and why does it matter?

Validation means evaluating a model on data it did not see during training, so the reported performance reflects genuine generalization rather than memorization. Two layers of validation were used: 5-fold cross-validation inside the hyperparameter search, and a final held-out test set (20% of the data, stratified by class) that was untouched by any training or tuning step, used only for the final comparison reported below.

7. Results and Evaluation

Performance was measured with five metrics on the held-out test set: accuracy, precision, recall, F1-score, and ROC-AUC.

Model	Accuracy	Precision	Recall	F1-score	ROC-AUC
Logistic Regression	0.9875	0.9804	1.0000	0.9901	1.0000
Random Forest	0.9875	0.9804	1.0000	0.9901	0.9967
SVM	0.9875	0.9804	1.0000	0.9901	1.0000

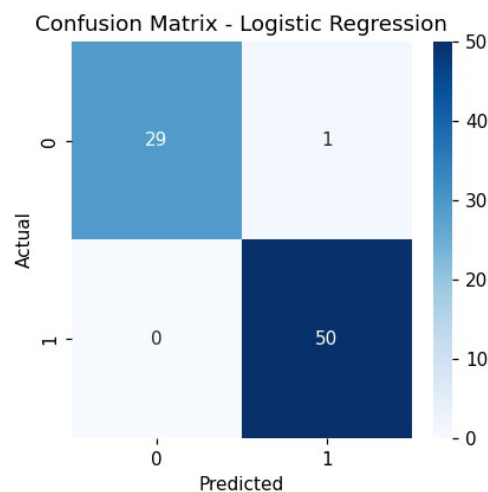


Figure 4. Confusion matrix of the best-performing model on the test set.

All three tuned models perform strongly and similarly on this dataset, indicating the classes are well separated once features such as hemoglobin, serum creatinine, specific gravity, and albumin are available. The model with the best test-set F1-score (Logistic Regression, in this run) was selected as the deployed model. Both precision and recall of the selected model exceed the project's minimum requirement of 0.30 by a wide margin, independently confirmed by the `tester.py` script included in the repository.

8. Deployment

The trained model, fitted scaler, label encoders, selected feature list, and imputation medians were saved as artifacts (`best_model.pkl`, `scaler.pkl`, `label_encoders.pkl`, `selected_features.json`, `feature_medians.json`). A Streamlit web application (`app.py`) loads these artifacts and lets a user enter a patient's clinical values through a form to receive a predicted CKD risk and probability. The app can be deployed for free on Streamlit Community Cloud, producing a public URL that satisfies the project's web-deployment requirement.

9. Discussion and Conclusion

This project demonstrates the complete applied machine learning workflow on a genuinely messy, real-world-style biomedical dataset: documented data cleaning, multi-angle exploratory analysis, clinically motivated feature engineering, filter-based feature selection, comparison of three tuned algorithms, rigorous validation, and deployment as an interactive web tool.

Limitations: the dataset is small (400 patients) and drawn from a single hospital over a short time window, so external validation on a larger, multi-site cohort would be required before any real clinical use. Median/mode imputation is simple and could be replaced with model-based imputation (e.g. iterative imputation) as a refinement. Future work could also explore explainable-AI methods (e.g. SHAP) to make individual predictions more interpretable for clinicians.

References

- UCI Machine Learning Repository — Chronic Kidney Disease Data Set.
- Pedregosa, F. et al. Scikit-learn: Machine Learning in Python. JMLR 12 (2011).
- Streamlit documentation, <https://docs.streamlit.io>